Building and Deploying a Sandboxed MCP File-Server Connector

Midterms Part 1 · Week 6: Model Context Protocol (MCP)

Submitted by
Jeth Sander S. Gecaro
BS3C

Subject
CSE2-M

Submitted to
Riga, Rasheed C.

October 15, 2026

Repository: github.com/gecarojeth-star/gecaro-mcp-file-server

1. Overview and Architecture

This project is a Model Context Protocol (MCP) server that lets AI agents, such as Claude, list, read, edit, search and version files inside one sandboxed folder on my laptop. It is reachable over a public HTTPS address and protected by a secret token. I used it to run a real multi-step task across two separate Claude chat sessions that shared context only through handoff files.

Claude -> HTTPS (ngrok) -> Docker -> auth -> tools -> sandbox -> workspace/

The code is split by responsibility. main.py builds the FastMCP server and wraps it in the auth middleware. config.py reads every setting from environment variables or a git-ignored .env file. sandbox.py resolves every path inside the workspace root. auth.py checks the token. audit.py logs every tool call to logs/calls.jsonl. errors.py turns bad input into clear messages. The tools live in tools/files.py, tools/search.py and tools/gitlite.py.

On first start, bootstrap.py creates the workspace, copies the four handoff templates into it (never overwriting real state) and makes it a git repository for the git tools.

2. Tech Stack and Why

Choice	**Why**
Python + official MCP SDK (FastMCP)	Type hints become strict JSON schemas and docstrings become tool descriptions, with little boilerplate.
Pinned mcp>=1.8,<2	SDK 2.x renamed FastMCP and broke my code. The pin keeps installs reproducible.
Streamable HTTP	Claude custom connectors need a remote URL. stdio only works on the same machine.
ngrok, free static domain	Free HTTPS with no router setup. The URL never changes across restarts.
Docker Compose	One command starts the server and tunnel. restart: unless-stopped brings both back after a crash or reboot. NSSM was the fallback, and it was not needed.

Restart proof. The public smoke test (scripts/check_remote.py) passed three times: after docker compose up, after docker compose restart, and after a full Windows reboot with Docker Desktop set to start on sign-in (evidence/docker_*_check.txt). Secrets stay in .env, which is never committed. Docker passes the ngrok token only to the tunnel container.

3. Tool Design

There are nine single-purpose tools. Each one does one job, so the agent always knows which tool to pick.

Tool	**Job**
list_files	List a folder (optionally recursive)
read_file	Read a file or a line range (capped at 20,000 chars)
write_file	Create or overwrite a file (atomic write)
str_replace	Replace exactly one match, else error
delete_file	Delete one file; needs confirm=true
search_code	Find text in a folder or a single file
git_status / git_commit / git_log	Lightweight version control of the workspace

- Clear descriptions. Each docstring says what the tool does, its limits, and when to use it (for example, "read a file before editing it").
- Strict schemas. Typed arguments with defaults, so wrong types are rejected before the tool runs.
- Helpful errors. Bad input returns isError with the next step, such as "Not deleted. Call delete_file again with confirm=true" or "path escapes the workspace root".
- Small outputs. Read, write, search and list results are all capped, which keeps agent calls cheap.
- Gating. delete_file needs confirm=true, deletes only single files, and refuses the four handoff files. .git is blocked entirely.

All tools were verified in MCP Inspector (all nine listed, and a ../secret.txt read refused), by the smoke-test script, and with a real client (Claude custom connector) on two Claude accounts.

4. Security and Sandboxing

Threat	Defense
Reading outside the folder	Rejects .., absolute paths, drive letters, null bytes and symlink escapes
Tampering with history	.git is unreadable and unwritable
Strangers on the public URL	Secret token in the URL path or a Bearer header, compared in constant time (hmac.compare_digest); all else gets 401
Weak setup	Refuses to start with a token under 24 chars, or with no token on a non-local host
Token in logs	Web access log turned off
Destructive mistakes	Gated delete; protected handoff files; every call logged
Container escape impact	Runs as a non-root user; port published only to 127.0.0.1

Tests. 39 unittest tests cover the sandbox, auth, tools and evidence script, and all pass (one symlink test is skipped on Windows, because creating symlinks needs extra rights). My own escape attempts were logged four times as "path escapes the workspace root".

Honest threats and tradeoffs

- Secret in the URL is weaker than a header. It can leak through screenshots, browser history or copied links. I chose it only because the Claude Free plan connector has no header field. During this project, an old token appeared in a screenshot, so I checked that it no longer works and covered it in the evidence.
- No code-execution tool, by design. A leaked token can only touch files in workspace/, never run commands on my laptop. The cost: agents cannot run tests.
- Free ngrok plan means usage limits and a third party in the request path.
- My laptop must stay on. If it sleeps, the connector goes offline.
- Anyone with the URL has full read/write access to the workspace. There are no per-user permissions.

5. Multi-Session Demo

Two agents worked in two separate Claude chats, with no shared chat history. The only shared context was the workspace and four handoff files: PROJECT_STATE.md (current state, read first), PLAN_LOG.md (append-only plan and result per session), CHECKPOINTS.md and DECISIONS.md. The server's instructions tell every agent to read the state first, read before editing, and update the files and git_commit before stopping.

	Agent A: Builder	**Agent B: Reviewer**
Job	Write a Python to-do CLI (add, list, done) and tests	Find bugs, fix with small edits, add tests
Logged calls	10	12
Self-reported	11	13
Mix	3 read, 3 write, 3 str_replace, 1 commit	5 read, 4 str_replace, 1 search, 1 write, 1 commit
Errors	0	1 (recovered)
Result	todo.py + 8 tests	Fixed crash on empty/corrupt file, atomic save, +7 tests (15)

Agent B started cold and picked up exactly where Agent A stopped, using only PROJECT_STATE.md and PLAN_LOG.md. Both sessions ended with a git commit in the workspace (evidence/workspace_git_log.txt).

Honest observations

- Self-reported vs logged. Each agent reported one more call than the server logged, most likely Claude's own "load tools" step, which never reaches the server.
- One failed call, not a blind retry. The Reviewer passed a file path to search_code, which then only accepted folders. Its next call took a different approach (read_file). I then changed search_code to accept single files.
- Whole-file reads. All 8 reads were whole-file reads. The files were small, so the cost was low, but line ranges would be better for larger files.
- Tests never ran. With no code-execution tool, neither agent could run the to-do tests. The Reviewer recorded this as an open problem in PROJECT_STATE.md.

6. Limitations and Next Steps

- Replace the URL secret with OAuth when a client supports it, or move to a header token on a paid plan.
- Add a safe, sandboxed test runner (for example, one container per run with no network) so agents can verify their work.
- Add per-session agent IDs to the audit log, so calls are split by agent instead of by time gaps.
- Encourage line-range reads, and add a file-size hint to list_files.
- Pin the ngrok image version and replace the deprecated --host-header flag with a traffic policy.
- Host the server on an always-on machine instead of my laptop.

Conclusion. The server meets the minimum requirements: a sandboxed root that rejects escapes, nine tools, HTTPS through a tunnel kept alive by Docker, handoff files that carried context across two sessions, and a real client connection, with honest notes on where it can improve.

Appendix: Evidence Screenshots

Screenshots are numbered in time order. Secrets and connector URLs are covered. Full-size copies are in evidence/screenshots/ in the repository.

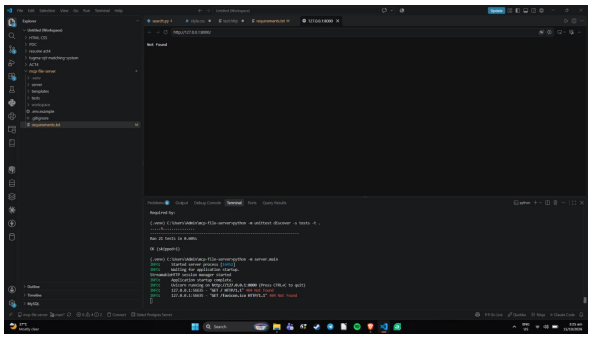


Figure 1. Unit tests passing and server starting (early milestone)

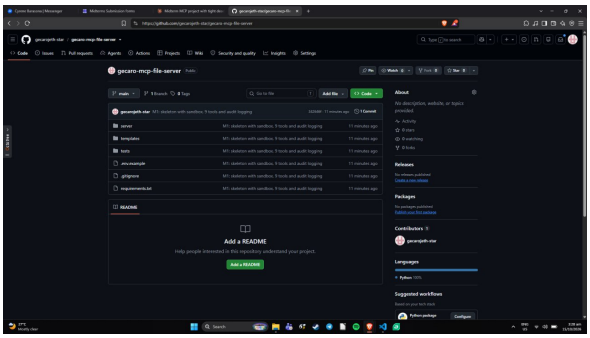


Figure 2. First commit pushed to the public GitHub repo

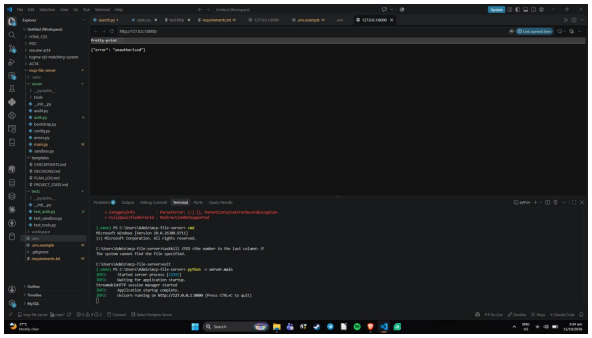


Figure 3. Local server refuses a request with no token

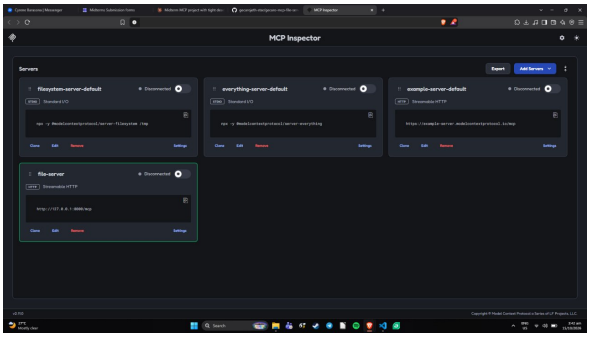


Figure 4. MCP Inspector: file-server added (Streamable HTTP)

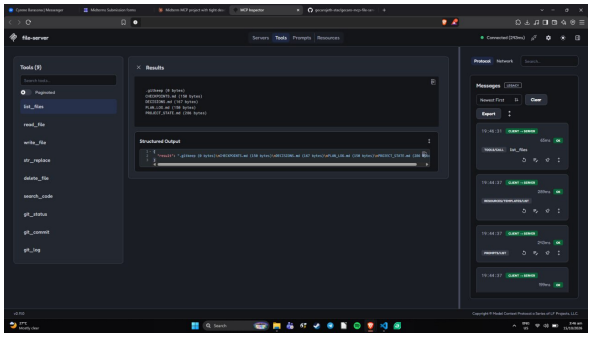


Figure 5. MCP Inspector: list_files returns the workspace

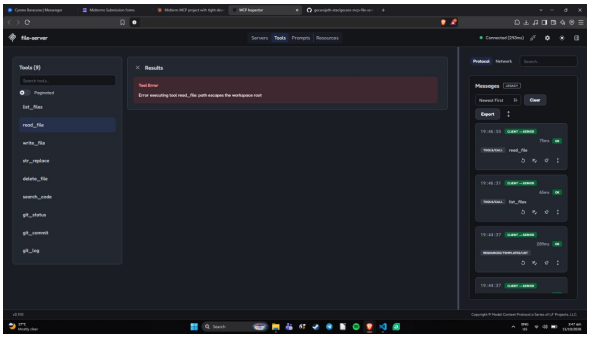


Figure 6. Inspector: 9 tools listed; ../secret.txt refused

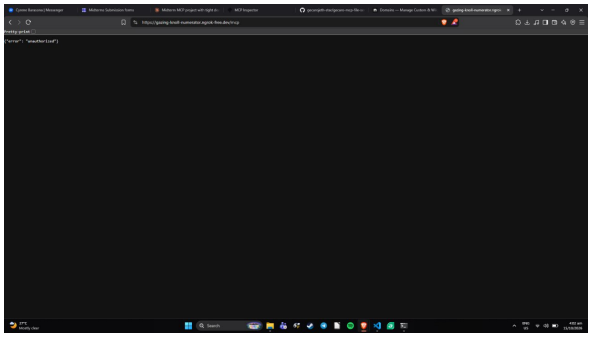


Figure 7. Public ngrok URL refuses a request with no token

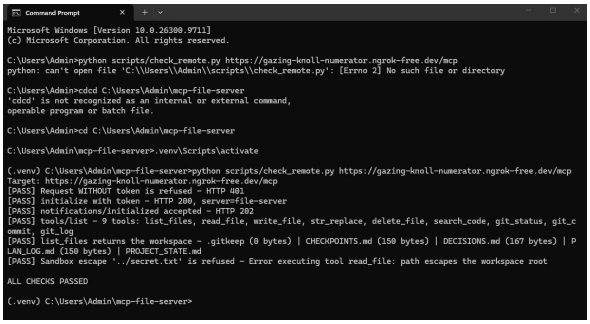


Figure 8. Remote smoke test over HTTPS: all checks passed

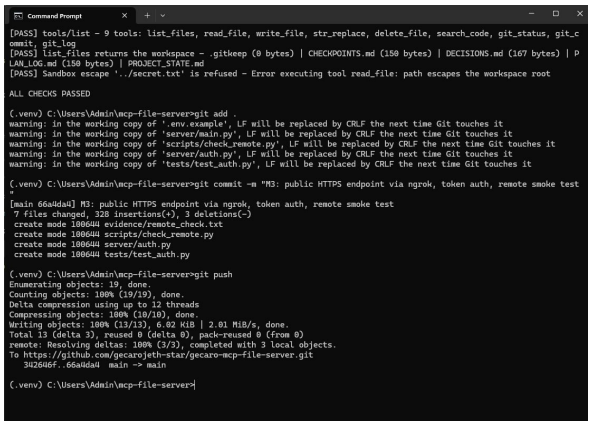


Figure 9. Milestone commit and push (meaningful git history)

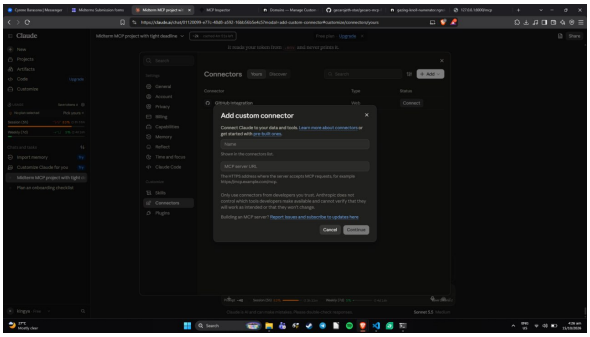


Figure 10. Adding the custom connector in Claude

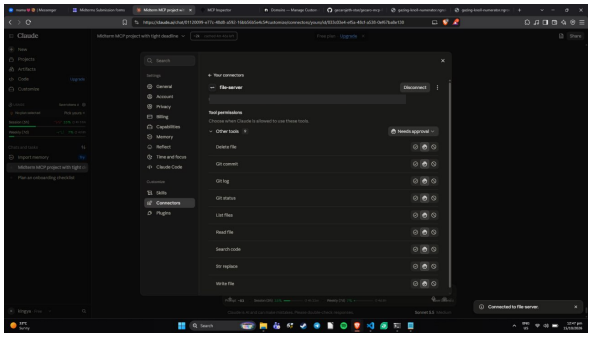


Figure 11. Claude connector shows all 9 tools (URL covered)

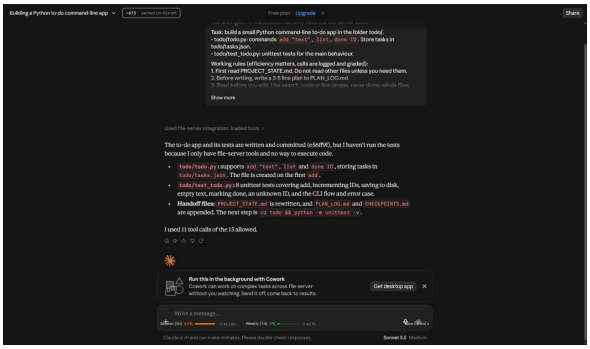


Figure 12. Session 1: Agent A (Builder) result

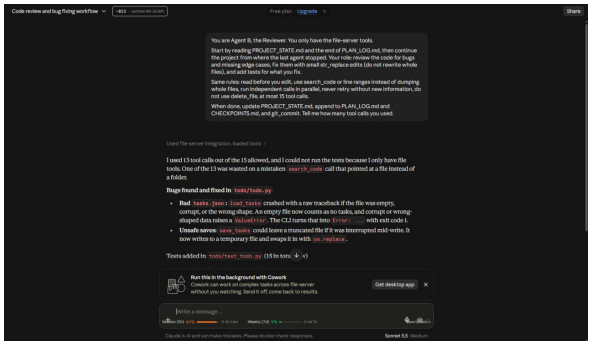


Figure 13. Session 2: Agent B (Reviewer) result

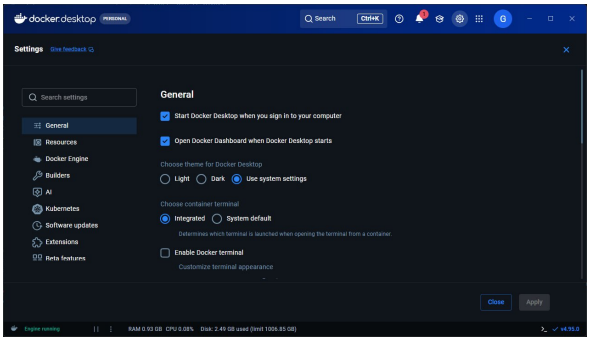


Figure 14. Docker Desktop starts on sign-in (survives reboot)

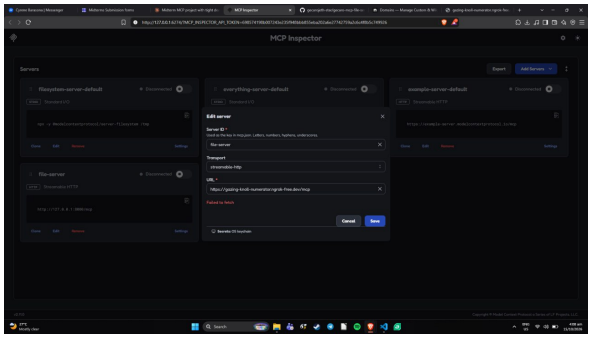


Figure 15. Claude (2nd account) reads the workspace via Docker

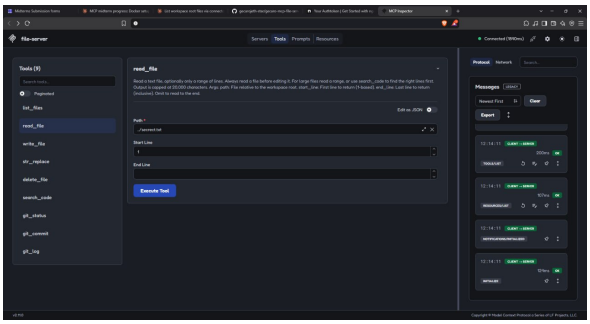


Figure 16. Inspector on Docker: escape attempt

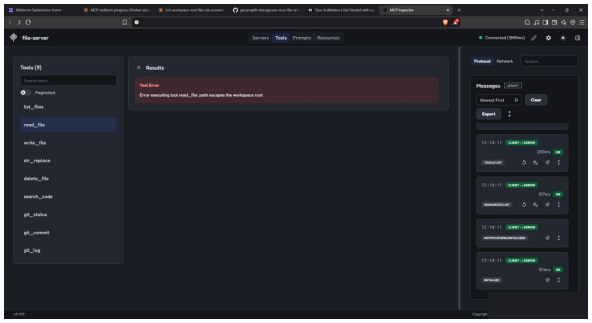


Figure 17. Inspector on Docker: escape refused